A Regression-Guided Superoptimizer for a Reduced RISC-V ISA

Pratyush Nandi
pratyush.nandi2024@vitstudent.ac.in

1. Introduction

Compilers translate high-level source code into machine instructions using optimization passes built from a fixed catalogue of hand-written rewrite rules, such as constant folding, strength reduction, and peephole substitution. While effective, this rule-based approach is fundamentally incomplete: it can only apply optimizations that a compiler engineer has explicitly anticipated and encoded, and it cannot discover non-obvious instruction sequences that a human would be unlikely to think of by hand. **Superoptimization** addresses this gap by reframing code optimization as a search problem: given a target instruction sequence, search the space of all instruction sequences that are provably equivalent to it, and return the one that is fastest, shortest, or otherwise cheapest under a chosen cost metric. Superoptimizers have historically been able to discover genuinely surprising, algorithmically distinct code that outperforms output from production compilers such as GCC and LLVM on small basic blocks.

This project proposes the design and implementation of a superoptimizer targeting a deliberately reduced, integer-only subset of the RISC-V instruction set architecture (RV32I), explicitly excluding all floating-point instructions and the integer division/remainder instructions from the M extension. This restriction is a deliberate scoping decision rather than a limitation: integer arithmetic without division has exact, total semantics — there are no rounding modes, no NaN values, and no division-by-zero edge cases to reason about — which makes correctness (equivalence) checking dramatically simpler and keeps the instruction and operand space small enough to search exhaustively for short sequences within a single academic term.

The central challenge in any superoptimizer is search cost: evaluating the true performance of a candidate instruction sequence, typically via a cycle-accurate simulator or real hardware execution, is slow, and a thorough search may need to evaluate anywhere from thousands to millions of candidate sequences. The core contribution proposed in this project is to replace this expensive evaluation step with a lightweight **regression model**, trained on a dataset of instruction sequences labeled with measured cost, that predicts a candidate sequence's cost directly from a small set of extracted features (instruction count, opcode histogram, data-dependency-chain depth, memory-operation count, and branch count). Embedding this learned cost model inside the search loop, in place of a full simulator call, is expected to make the search dramatically faster while remaining an accurate-enough proxy for real performance to guide the search toward genuinely better code.

The system will combine classical search strategies from the superoptimization literature — enumerative search over short sequences, and stochastic (Markov Chain Monte Carlo) search over longer ones — with this regression-based cost model, and will be evaluated by comparing the instruction count and estimated execution cost of superoptimized output against GCC/Clang -O0 and -O2 output on a benchmark set of loop-free basic blocks. The project is deliberately scoped below the

complexity of recent large language-model-based superoptimizers, prioritizing a small, fully explainable, and rigorously verifiable system over a large opaque one, which is appropriate both for the timeline of a course project and for producing results that can be reasoned about and verified by hand.

2. Literature Survey

The comparative analysis of recent studies on superoptimization and learned cost models relevant to this project is provided below.

Schkufza, Sharma, and Aiken [1] introduced STOKE, a stochastic superoptimizer that reformulates loop-free binary optimization as a cost-minimization problem solved via Markov Chain Monte Carlo (MCMC) search rather than exhaustive enumeration. Evaluated on benchmarks drawn from cryptography, linear algebra, and low-level bit-manipulation routines, STOKE was able to produce x86-64 code that matched or outperformed the output of production compilers such as GCC at the -O3 optimization level, including discovering hardware-intrinsic-based rewrites that conventional compiler passes did not find.

Mendis, Renda, Amarasinghe, and Carbin [2] proposed Ithemal, a learned cost model that replaces hand-engineered analytical simulators with a data-driven regression approach for predicting the throughput of x86-64 basic blocks. Trained on the BHive dataset of roughly 300,000 real hardware-measured basic blocks across three Intel microarchitectures, Ithemal used a two-level recurrent neural network to map instruction sequences directly to a predicted throughput value, reducing mean absolute percentage error by more than 50% compared to established analytical tools such as LLVM's llvm-mca and Intel's IACA, while remaining fast enough to be queried repeatedly inside a search or optimization loop.

Wei et al. [3] proposed SuperCoder, which investigates whether large language models can act as superoptimizers by fine-tuning them with reinforcement learning (PPO and GRPO) against a reward combining program correctness and measured runtime speedup. Built on the first large-scale superoptimization benchmark of over 8,000 assembly programs (far larger than the short, loop-free programs used in most prior work), the fine-tuned SuperCoder model achieved 95.0% correctness with an average 1.46x speedup over GCC -O3 output, demonstrating that learned, search-free approaches can compete with and extend classical search-based superoptimization at much larger program scale.

Bansal and Aiken [4] developed an enumerative peephole superoptimizer that exhaustively searches short x86 instruction sequences and, using a hybrid testing-then-solver verification strategy, builds a reusable database of provably optimal rewrites. Applied to code generated by production compilers, the resulting peephole optimizer produced measurable reductions in both instruction count and runtime over the unoptimized compiler output, establishing enumerative search combined with automated equivalence checking as a practical, soundness-preserving alternative to purely hand-written peephole rules.

Phothilimthana, Thakur, Bodik, and Dhurjati [5] introduced GreenThumb, a superoptimizer construction framework that combines enumerative, stochastic, and SMT-based search strategies behind a single retargetable interface, so that supporting a new instruction set architecture requires

implementing only an emulator for it rather than a new search engine. Evaluated across multiple ISAs including ARM and a subset of LLVM IR, GreenThumb demonstrated that hybrid search strategies scale better than any single strategy alone, directly informing this project's decision to combine enumerative and stochastic search behind a common regression-guided cost model.

Table 1: Literature Survey

S.No.	Title & Year	Dataset Used	Algorithm Used	Performance Measure	Future Work
1	Stochastic Superoptimization (Schkufza, Sharma & Aiken, ASPLOS 2013)	Benchmarks from cryptography, linear algebra, and low-level programming (x86-64 binaries; later extended to the 25-program "Hacker's Delight" suite)	Stochastic search via Markov Chain Monte Carlo (MCMC) sampling, guided by a hand-defined cost function balancing correctness and performance	Runtime performance vs. gcc -O3 / icc -O3 output; whether STOKE output matches or outperforms production-compiler and expert hand-written code	The uniform, hand-fixed proposal distribution used to sample mutations does not learn from past search behaviour or leverage program semantics, which follow-up work identifies as the main limitation to build on (e.g. learning the proposal distribution itself).
2	Ithemal: Accurate, Portable and Fast Basic Block Throughput Estimation using Deep Neural Networks (Mendis, Renda, Amarasinghe & Carbin, ICML 2019)	BHive — approx. 300,000 x86-64 basic blocks labeled with real hardware throughput on Haswell, Skylake, and Ivy Bridge microarchitectures	Hierarchical multiscale recurrent neural network (two-level LSTM) trained as a regression model to predict basic-block throughput	Mean Absolute Percentage Error (MAPE) between predicted and measured throughput; reduced MAPE by more than 50% relative to llvm-mca and IACA	A separate model instance must be retrained for every microarchitecture, and the model outputs only a throughput prediction with no explanation of which part of the block drives the cost, limiting its reliability for direct use in mainstream compiler optimization pipelines.
3	SuperCoder: Assembly Program Superoptimization with Large Language Models (Wei et al., 2025)	First large-scale superoptimization benchmark: 8,072 assembly programs (avg. 130 lines), versus 2–15-line programs in prior datasets	LLM fine-tuned with reinforcement learning (PPO and GRPO) using a reward function combining program correctness and measured speedup	Test-passing / correctness rate and average speedup vs. gcc -O3; fine-tuned model reached 95.0% correctness with 1.46x average speedup	Establishing this as a foundation for further research into program performance optimization beyond traditional compiler heuristics, including scaling to larger and more diverse real-world programs beyond the current benchmark.
4	Automatic Generation of Peephole Superoptimizers (Bansal & Aiken, ASPLOS 2006)	x86 instruction sequences and code fragments generated by production compilers, used to build a database of optimal short rewrites	Exhaustive enumerative search over short instruction sequences, with a hybrid testing-then-solver strategy for equivalence checking	Reduction in instruction count and measured runtime of compiler-generated code after applying the learned peephole database	Extending the enumerative database approach to longer sequences and to additional instruction sets beyond x86, and reducing reliance on the solver fallback to improve scalability of the verification step.

S.No.	Title & Year	Dataset Used	Algorithm Used	Performance Measure	Future Work
5	GreenThumb: Superoptimizer Construction Framework (Phothilimthana, Thakur, Bodik & Dhurjati, UCB/EECS-2016-8)	Programs across multiple target ISAs, including ARM and a subset of LLVM IR, used to evaluate the retargetable framework	Hybrid framework combining enumerative, stochastic (MCMC), and SMT-based search strategies behind a common retargetable interface	Search scalability and quality of optimizations found, compared across individual search strategies used alone versus combined	Making the framework retargetable to new ISAs by implementing only an emulator, and further improving how the hybrid strategies are scheduled and combined during a single search run.

Note: Future Work entries are paraphrased summaries of each paper's stated limitations / future directions, drawn from the paper itself (SuperCoder) or from how follow-up literature characterizes the work's open limitations (STOKE, Ithemal). Copyright limits on exact quotation mean these are not verbatim excerpts — replace with the literal copy-pasted sentence(s) from each PDF if your submission requires exact text, keeping the yellow highlight.

3. References

[1] E. Schkufza, R. Sharma, and A. Aiken, “Stochastic Superoptimization,” in Proc. 18th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), Houston, TX, USA, Mar. 2013, pp. 305–316, doi: 10.1145/2451116.2451150.

[2] C. Mendis, A. Renda, S. Amarasinghe, and M. Carbin, “Ithemal: Accurate, Portable and Fast Basic Block Throughput Estimation using Deep Neural Networks,” in Proc. 36th Int. Conf. Machine Learning (ICML), Long Beach, CA, USA, Jun. 2019, pp. 4505–4515.

[3] A. Wei, T. Suresh, H. Tan, Y. Xu, G. Singh, K. Wang, and A. Aiken, “SuperCoder: Assembly Program Superoptimization with Large Language Models,” arXiv:2505.11480, May 2025.

[4] S. Bansal and A. Aiken, “Automatic Generation of Peephole Superoptimizers,” in Proc. 12th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), San Jose, CA, USA, Oct. 2006, pp. 394–403, doi: 10.1145/1168857.1168906.

[5] P. M. Phothilimthana, A. Thakur, R. Bodik, and D. Dhurjati, “GreenThumb: Superoptimizer Construction Framework,” Univ. California, Berkeley, Tech. Rep. UCB/EECS-2016-8, Feb. 2016.

[6] H. Massalin, “Superoptimizer: A Look at the Smallest Program,” in Proc. 2nd Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), Palo Alto, CA, USA, Oct. 1987, pp. 122–126, doi: 10.1145/36206.36194.

[7] R. Sasnauskas, Y. Chen, P. Collingbourne, J. Ketema, G. Taneja, and J. Regehr, “Souper: A Synthesizing Superoptimizer,” arXiv:1711.04422, Nov. 2017.

[8] Z. Chen, Y. Yang, A. Guo, and T. Chen, “Ansor: Generating High-Performance Tensor Programs for Deep Learning,” in Proc. 14th USENIX Symp. Operating Systems Design and Implementation (OSDI), Nov. 2020, pp. 863–879.

[9] O. Sykora, P. Phothilimthana, C. Mendis, and A. Yazdanbakhsh, “GRANITE: A Graph Neural Network Model for Basic Block Throughput Estimation,” arXiv:2210.03894, Oct. 2022.

[10] COMET: A Neural Cost Model Explanation Framework, in Proc. 7th Conf. Machine Learning and Systems (MLSys), arXiv:2302.06836, 2024.

[11] A. Wei, T. Suresh, and A. Aiken, “Pearl: Automatic Code Optimization using Deep Reinforcement Learning,” in Proc. 39th ACM Int. Conf. Supercomputing (ICS), 2025.

[12] A. Waterman and K. Asanovic, Eds., “The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA,” RISC-V International, 2019.

** Note: Create a paper folder. In that folder, save all pdf files as year_title of the paper.*